63517 Informations- und Kodierungstheorie – L2: Codierungen, Codes und Information

Inhalt

3 Codierungen, Codes und Information
- 3.1 Motivation
- 3.2 Definition und Eigenschaften
- 3.3 Längenvariable Codes
  - 3.3.1 Präfixfreie Codes
  - 3.3.2 Kraft'sche Ungleichung
- 3.4 Blockcodes
  - 3.4.1 Zeichencodes
  - 3.4.2 Zahlencodes
  - 3.4.3 Lineare Codes
- 3.5 Der Übertragungskanal
  - 3.5.1 Kanalkapazität
  - 3.5.2 Kapazität des Morse-Kanals
- 3.6 Der Informationsbegriff

Vollständige Zusammenfassung pro Abschnitt.
Quelle: 63517 Informations- und Kodierungstheorie, Lektion 2 = Kapitel 3 des Basistexts (Hoffmann, Einführung in die Informations- und Codierungstheorie, 2. Auflage), Ausgabe WiSe 2026/27.
Gliederung und Nummerierung entsprechen der Quelle; einzelne Seitenverweise beziehen sich auf diese Ausgabe.
Nicht enthalten: Kapitelvorspann, Kalenderblatt zu Louis Braille, Fotos und Illustrationen (Abb. 3.12, 3.13, 3.15, 3.17, 3.36, 3.39, 3.43), 3.7 Übungsaufgaben.

3 Codierungen, Codes und Information

Das Kapitel legt die Grundbegriffe fest: Was eine Codierung und was ein Code ist (3.2), die beiden Codeklassen längenvariable Codes (3.3) und Blockcodes (3.4) mit Beispielen aus der Praxis und den linearen Codes, den Übertragungskanal und seine Kapazität nach Shannon (3.5) und schließlich den Informationsbegriff (3.6).

3.1 Motivation

Überblick über das Kapitel: In 3.2 werden **Codierung** und **Code** definiert, in 3.3 und 3.4 die wichtigsten Codeklassen vorgestellt, in 3.5 werden im Shannon'schen Übertragungsmodell **Übertragungsrate** und **Kanalkapazität** definiert, 3.6 diskutiert den Shannon'schen Informationsbegriff.

3.2 Definition und Eigenschaften

Definition 3.1 (Codierung)

Σ und Π seien zwei endliche Mengen von Grundsymbolen. Unter einer *Codierung* verstehen wir eine injektive Abbildung der Form

$$c : \Sigma^+ \to \Pi^+$$

Σ ist das *Quellenalphabet* und Π das *Codealphabet* von c.

Σ^+ und Π^+ enthalten alle endlichen Sequenzen aus Symbolen von Σ bzw. Π mit mindestens einem Zeichen:

$$\Sigma^+ := \{\sigma_1\sigma_2\ldots\sigma_n \mid n \geq 1,\ \sigma_i \in \Sigma\} \qquad \Pi^+ := \{\tau_1\tau_2\ldots\tau_n \mid n \geq 1,\ \tau_i \in \Pi\}$$

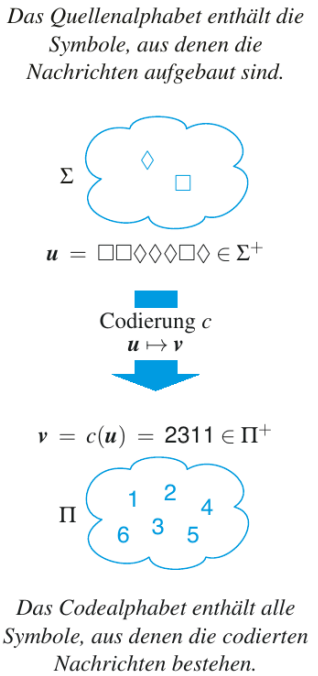


Abbildung 3.1: Eine Codierung c ist eine Abbildung, die jede Nachricht $\boldsymbol{u} \in \Sigma^+$ auf ein Codewort $\boldsymbol{v} \in \Pi^+$ abbildet.

- Mit dem Codealphabet $\Pi = \{0, 1\}$ spricht man von **binären Codierungen**. Fast alle Codierungen im Buch sind binär oder zu einer binären Codierung äquivalent.
- Typische Quellenalphabete: $\{0, 1\}$ (Codierungen, die Binärsequenzen entgegennehmen) und $\{0, \ldots, 255\}$ (Codierungen, die einen Datenstrom Byte für Byte übersetzen).
- **Injektivität**: $c(\boldsymbol{u}_1) = c(\boldsymbol{u}_2) \Rightarrow \boldsymbol{u}_1 = \boldsymbol{u}_2$. Die Originalnachricht ist durch die codierte Sequenz eindeutig bestimmt und lässt sich eins zu eins rekonstruieren; das Buch behandelt deshalb ausschließlich **verlustfreie** Codierungen.
- **Verlustbehaftete** Codierungen (Kompressionsformate für Audio und Video, die bewusst auf Information verzichten) sind ein eigenes Forschungsfeld und werden nicht behandelt.

Lauflängencodierung

Bei der **Lauflängencodierung** (*Run-Length Encoding*, kurz RLE) werden mehrfache Wiederholungen des gleichen Quellensymbols durch einen Zähler ausgedrückt. Zwei Varianten, am Beispiel des Quellenalphabets $\Sigma = \{\mathsf{A}, \ldots, \mathsf{Z}\}$ und des Worts KAFFEE:

Variante	Beispiel	Eigenschaften
ohne Zählerzeichen	KAFFEE ↦ 1K1A2F2E	Jedem Zeichen wird ein Zähler vorangestellt. Die Position sagt, ob Zeichen oder Lauflänge – erleichtert die Decodierung. Nachteil: erhebliche Verlängerung, wenn sich nur wenige Symbole wiederholen.
mit Zählerzeichen	KAFFEE ↦ KA#2F#2E	Jedem Zähler wird ein Markierungssymbol (z. B. #) vorangestellt, um Verwechslung mit Quellensymbolen zu vermeiden. Überlegen, wenn sich nur wenige Symbole in Ketten wiederholen; setzt ein freies Symbol als Markierungszeichen voraus.

Besonders effizient ist die Lauflängencodierung bei binärem Quellenalphabet, z. B. für Schwarz-Weiß-Bilder: Da sich die beiden Symbole stets abwechseln, genügt es, nur die Lauflängen zu speichern.

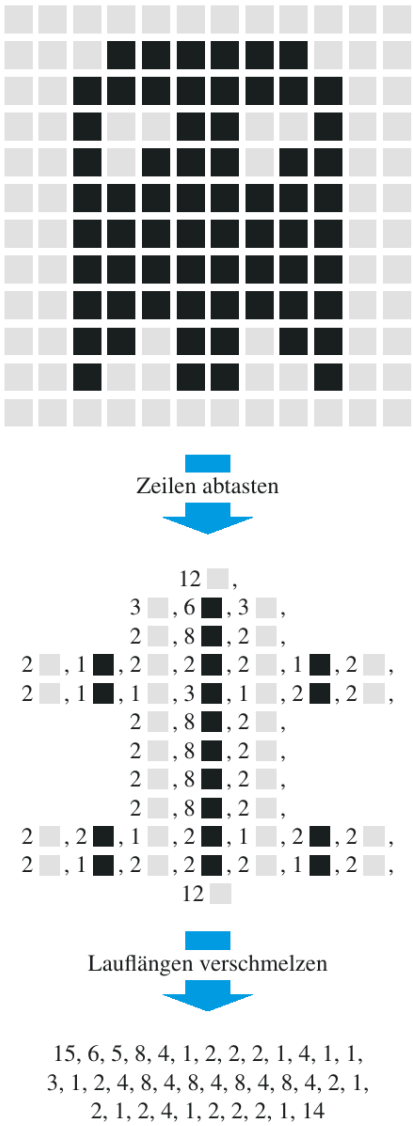


Abbildung 3.2: Lauflängencodierung am Beispiel einer binären Nachricht

Das 12 × 12-Pixelbild wird zeilenweise abgetastet; die Lauflängen über die Zeilengrenzen hinweg zusammengefasst ergeben die Folge 15, 6, 5, 8, 4, 1, 2, 2, 2, 1, 4, 1, 1, 3, 1, 2, 4, 8, 4, 8, 4, 8, 4, 8, 4, 2, 1, 2, 1, 2, 4, 1, 2, 2, 2, 1, 14 (beginnend mit Weiß).

Zeichenweise Codierung

Viele Codierungen ordnen jedem Element des Quellenalphabets eine Sequenz von Codesymbolen zu und codieren eine Nachricht $\boldsymbol{u} = u_0 u_1 \dots u_{n-1}$ zeichenweise:

$$c(\boldsymbol{u}) = c(u_0 u_1 \dots u_{n-1}) = c(u_0)\, c(u_1) \dots c(u_{n-1}) \tag{3.1}$$

Definition 3.2 (Code, Codewort)

Sei c eine Codierung. Der von c erzeugte *Code* ist die Menge

$$C := c(\Sigma) := \{c(\sigma) \mid \sigma \in \Sigma\}$$

Die Zeichenkette $c(\sigma)$ ist das *Codewort* des Zeichens σ.

Zwei Darstellungen zeichenweise arbeitender Codierungen:

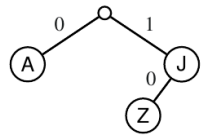
- **Tabellarische Darstellung**: Symbole des Quellenalphabets mit ihren Codewörtern.
- **Baumdarstellung**: Jeder Pfad von der Wurzel zu einem Knoten beschreibt ein mögliches Codewort. Bei $\Pi = \{0, 1\}$ entsteht ein Binärbaum; Konvention: Kante nach links = `0`, Kante nach rechts = `1`.

■ Codierung c_1

Tabellarische Darstellung:

σ	$c_1(\sigma)$
A	0
J	1
Z	10

Baumdarstellung:



■ Codierung c_2

Tabellarische Darstellung:

σ	$c_2(\sigma)$
A	0
J	10
Z	11

Baumdarstellung:

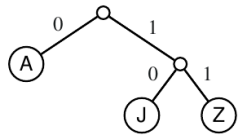


Abbildung 3.3: Zwei Darstellungsmöglichkeiten für zeichenweise arbeitende Codierungen

Die beiden Beispielcodierungen über $\Sigma = \{A, J, Z\}$: c_1: A ↦ `0`, J ↦ `1`, Z ↦ `10`; c_2: A ↦ `0`, J ↦ `10`, Z ↦ `11`.

Codes zeichenweise arbeitender Codierungen werden in **längenvariable Codes** und **Blockcodes** unterschieden.

3.3 Längenvariable Codes

Ein Code oder eine Codierung ist **längenvariabel**, wenn mindestens zwei Codewörter unterschiedlich lang sind. Einsatz vor allem in der Datenkompression: häufige Symbole erhalten kürzere, seltene längere Sequenzen.

c_1 und c_2 aus Abbildung 3.3 sind beide längenvariabel.

Beispiel: Codierung der Nachricht JAZZ (Abb. 3.4)

- c_1: JAZZ ↦ `1` `0` `10` `10` = `101010`
- c_2: JAZZ ↦ `10` `0` `11` `11` = `1001111`

3.3.1 Präfixfreie Codes

Unterschied der beiden Codierungen zeigt sich beim Decodieren: Mit c_2 ist eine Symbolsequenz immer eindeutig decodierbar, mit c_1 nicht. Ohne Trennzeichen bilden unter c_1 alle folgenden Nachrichten auf dieselbe Sequenz ab (Abb. 3.5):

ZZZ, JAZZ, ZJAZ, ZZJA, JAJAZ, JAZJA, ZJAJA, JAJAJA ↦ `101010`

Bei c_2 ist kein Codewort das Anfangsstück eines anderen.

Definition 3.3 (Präfixfreier Code, Fano-Bedingung)

Ist kein Codewort der Anfang eines anderen Codeworts, so

- ist der Code *präfixfrei* und
- erfüllt die *Fano-Bedingung*.

Kriterium am Codebaum: Ein Code ist genau dann präfixfrei, wenn kein innerer Knoten mit einem Symbol des Quellenalphabets markiert ist.

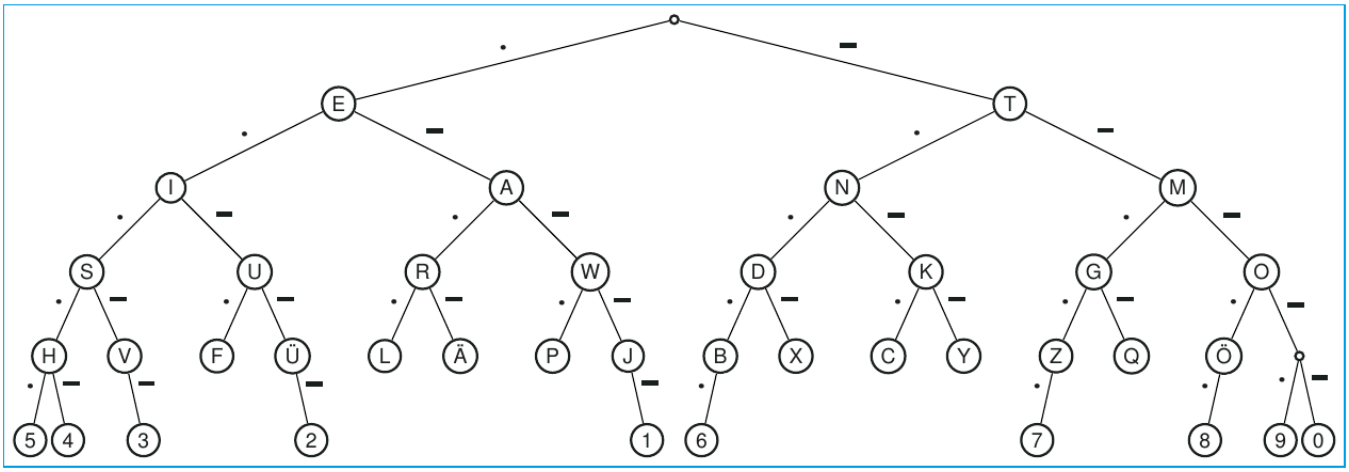


Abbildung 3.6: Baumdarstellung des Morse-Codes (ohne Sonderzeichen). Der Code ist nicht präfixfrei, da neben den Blättern auch innere Knoten mit Quellensymbolen markiert sind.

In der Morse-Telegrafie wird die fehlende Präfixfreiheit durch Pausen nach jedem Buchstaben und jedem Wort kompensiert – sinnvoll, wenn ein Mensch empfängt. Bei automatischer Decodierung ist Präfixfreiheit von Vorteil; dort werden fast ausschließlich Codes ohne Pausenzeichen verwendet.

Beispiel: Faxcodierung

Viele praxisrelevante Verfahren übersetzen stufenweise mit verschiedenen Codierungsverfahren. Telefaxcodierung in zwei Schritten:

1. Die Vorlage wird Zeile für Zeile abgetastet und für jede Pixelkette die Länge berechnet (Lauflängencodierung wie in Abbildung 3.2).
2. Die Lauflängen werden durch die Bitmuster aus Tabelle 3.1 dargestellt. Wie bei allen längenvariablen Codierungen erhalten häufige Symbole (hier: Lauflängen) kurze, seltene lange Bitsequenzen. Da weiße und schwarze Pixelketten unterschiedlich häufig sind, gibt es für jede Farbe eine eigene Tabelle.

Die Übertragung beginnt immer mit einer weißen Pixelkette; abwechselnd wird in der Spalte für weiße und schwarze Lauflängen nachgeschlagen (Abb. 3.7 zeigt den Bitstrom für die Grafik aus Abbildung 3.2). Weiße und schwarze Lauflängen sind jeweils präfixfrei codiert, sodass sich die Codewörter eindeutig aus dem Bitstrom extrahieren lassen.

Tabelle 3.1: Lauflängencodierung am Beispiel des Faxprotokolls

n	Weiß	Schwarz	n	Weiß	Schwarz	n	Weiß	Schwarz
0	00110101	0000110111	32	00011011	000001101010	64	11011	0000001111
1	000111	010	33	00010010	000001101011	128	10010	000011001000
2	0111	11	34	00010011	000011010010	192	010111	000011001001
3	1000	10	35	00010100	000011010011	256	0110111	000001011011
4	1011	011	36	00010101	000011010100	320	00110110	000000110011
5	1100	0011	37	00010110	000011010101	384	00110111	000000110100
6	1110	0010	38	00010111	000011010110	448	01100100	000000110101
7	1111	00011	39	00101000	000011010111	512	01100101	0000001101100
8	10011	000101	40	00101001	000001101100	576	01101000	0000001101101
9	10100	000100	41	00101010	000001101101	640	01100111	0000001001010
10	00111	0000100	42	00101011	000011011010	704	011001100	0000001001011
11	01000	0000101	43	00101100	000011011011	768	011001101	0000001001100
12	001000	0000111	44	00101101	000001010100	832	011010010	0000001001101
13	000011	00000100	45	00000100	000001010101	896	011010011	0000001110010
14	110100	00000111	46	00000101	000001010110	960	011010100	0000001110011
15	110101	000011000	47	00001010	000001010111	1024	011010101	0000001110100
16	101010	0000010111	48	00001011	000001100100	1088	011010110	0000001110101
17	101011	0000011000	49	01010010	000001100101	1152	011010111	0000001110110
18	0100111	0000001000	50	01010011	000001010010	1216	011011000	0000001110111
19	0001100	00001100111	51	01010100	000001010011	1280	011011001	0000001010010
20	0001000	00001101000	52	01010101	000000100100	1344	011011010	0000001010011
21	0010111	00001101100	53	00100100	000000110111	1408	011011011	0000001010100
22	0000011	00000110111	54	00100101	000000111000	1472	010011000	0000001010101
23	0000100	00000101000	55	01011000	000000100111	1536	010011001	0000001011010
24	0101000	00000010111	56	01011001	000000101000	1600	010011010	0000001011011
25	0101011	00000011000	57	01011010	000001011000	1664	011000	0000001100100
26	0010011	000011001010	58	01011011	000001011001	1728	010011011	0000001100101
27	0100100	000011001011	59	01001010	000000101011	1792	00000001000	00000001000

n	Weiß	Schwarz	n	Weiß	Schwarz	n	Weiß	Schwarz
28	0011000	000011001100	60	01001011	000000101100	1856	00000001100	00000001100
29	00000010	000011001101	61	00110010	000001011010	1920	00000001101	00000001101
30	00000011	000001101000	62	00110011	000001100110	1984	000000010010	000000010010
31	00011010	000001101001	63	00110100	000001100111	EOL	000000000001	000000000001

Nicht jede Lauflänge hat ein Codewort. Längere Ketten werden zerlegt:

Beispiel: Ketten ohne eigenes Codewort

- **68 weiße Pixel** = 64 weiß, 0 schwarz, 4 weiß ↦ `11011` `0000110111` `1011`
- **Erstes Pixel schwarz** (68 schwarze Pixel): Da die Übertragung mit Weiß beginnen muss, wird zuerst das Bitmuster für die weiße Lauflänge 0 gesendet: 0 weiß, 64 schwarz, 0 weiß, 4 schwarz ↦ `00110101` `0000001111` `00110101` `011`

3.3.2 Kraft'sche Ungleichung

Codebäume präfixfreier Codes lassen sich als mehrstufiges Zufallsexperiment deuten: An der Wurzel starten, eine Münze werfen, je nach Kopf oder Zahl zum linken oder rechten Kind gehen, bis ein Blatt erreicht ist; dessen Symbol ist das Ergebnis.

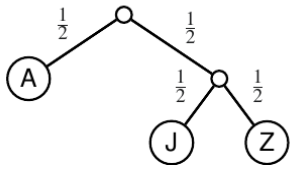


Abbildung 3.8: Der Codebaum eines präfixfreien Codes lässt sich als die Beschreibung eines mehrstufigen Wahrscheinlichkeitsexperiments interpretieren. Die Kanten sind mit Eintrittswahrscheinlichkeiten markiert.

Für die Codierung c_2 (A ↦ `0`, J ↦ `10`, Z ↦ `11`) mit den Codewortlängen l_1, l_2, l_3 von $c(\mathsf{A}), c(\mathsf{J}), c(\mathsf{Z})$:

$$p(\mathsf{A}) = \frac{1}{2} = \frac{1}{2^{l_1}}, \quad p(\mathsf{J}) = \frac{1}{2}\cdot\frac{1}{2} = \frac{1}{4} = \frac{1}{2^{l_2}}, \quad p(\mathsf{Z}) = \frac{1}{2}\cdot\frac{1}{2} = \frac{1}{4} = \frac{1}{2^{l_3}}$$

Da mit Wahrscheinlichkeit 1 eines der Symbole erreicht wird:

$$\sum_{i=1}^{3} \frac{1}{2^{l_i}} = 1 \tag{3.2}$$

Verallgemeinerung mit zwei Besonderheiten (Abb. 3.9):

- Allgemein ist der Codebaum kein Binärbaum, sondern innere Knoten haben bis zu $|\Pi|$ Nachfolger; jedes Codewort geht mit dem Gewicht $\frac{1}{|\Pi|^{l_i}}$ in die Summe ein.
- Der Codebaum muss nicht saturiert sein (innere Knoten mit weniger als $|\Pi|$ Kindern); dann führt nicht jeder Ausgang zu einem Symbol, und aus „=“ wird „≤“.

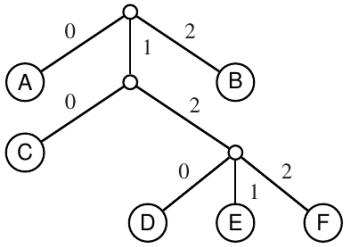


Abbildung 3.9: Codebaum eines präfixfreien Codes mit einem dreielementigen Codealphabet

Codierung c_3 mit $\Pi = \{0, 1, 2\}$: A ↦ `0`, B ↦ `2`, C ↦ `10`, D ↦ `120`, E ↦ `121`, F ↦ `122`. Der Baum ist nicht saturiert (der Knoten `1` hat kein Kind `11`).

Damit erfüllt jeder präfixfreie Code mit Codewortlängen l_i über dem Codealphabet Π:

$$\sum_i \frac{1}{|\Pi|^{l_i}} \leq 1 \tag{3.3}$$

Satz 3.1 (Kraft'sche Ungleichung, Leon Kraft, 1949)

Für jede Folge $l_1, \ldots, l_n$ von natürlichen Zahlen gilt:

Es existiert ein präfixfreier Code mit dem Codealphabet Π und den Codewortlängen $l_1, \ldots, l_n \Leftrightarrow \sum_i \frac{1}{|\Pi|^{l_i}} \leq 1$

Die Überlegung oben zeigt die Richtung von links nach rechts. Der Satz gilt auch in die andere Richtung: Erfüllen die Längen die Ungleichung, so existiert ein präfixfreier Code mit diesen Codewortlängen.

Konstruktion eines präfixfreien Codes (Rückrichtung)

Gegeben: binäres Codealphabet ($|\Pi| = 2$) und eine Folge von n natürlichen Zahlen $l_1, \ldots, l_n$ mit $\sum_i 2^{-l_i} \leq 1$, absteigend sortiert ($l_i \geq l_{i+1}$).

1. Summen bilden:

$$s_1 := 0 \tag{3.4}$$

$$s_{i+1} := s_i + 2^{-l_{i+1}} \tag{3.5}$$

2. Die Werte s_i in Binärdarstellung aufschreiben.
3. Von der i-ten Zahl die ersten l_i Nachkommaziffern als Codewort nehmen.

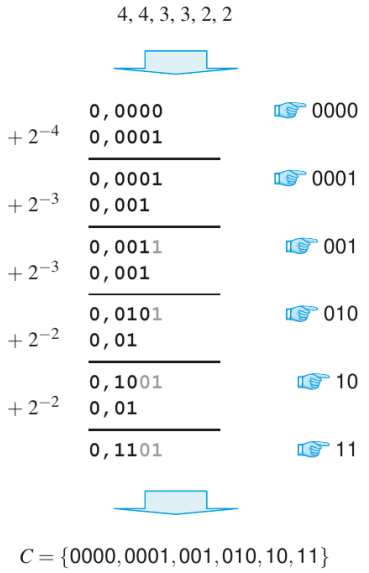


Abbildung 3.10: Konstruktion eines präfixfreien Codes, dessen Codewörter die vorgegebenen Längen aufweisen.

Beispiel Längen 4, 4, 3, 3, 2, 2 ($2 \cdot 2^{-4} + 2 \cdot 2^{-3} + 2 \cdot 2^{-2} = \frac{7}{8} \leq 1$): $s_1, \ldots, s_6$ = 0,0000; 0,0001; 0,0011; 0,0101; 0,1001; 0,1101 ergeben $C = \{0000, 0001, 001, 010, 10, 11\}$.

Beweis der Präfixfreiheit. Aus (3.5) folgt für $1 \leq j \leq n - i$

$$s_{i+j} = s_{i+j-1} + 2^{-l_{i+j}} \geq s_i + 2^{-l_{i+j}}, \quad \text{also} \quad s_{i+j} - s_i \geq 2^{-l_{i+j}} \tag{3.6}$$

Angenommen, der Code wäre nicht präfixfrei. Dann gibt es s_i und s_{i+j}, sodass s_{i+j} in den ersten l_{i+j} Nachkommastellen mit s_i übereinstimmt. Die Voraussetzung $\sum_i 2^{-l_i} \leq 1$ sorgt dafür, dass die Vorkommaanteile beider Zahlen 0 sind. Dann liegen sie so nahe beieinander, dass $s_{i+j} - s_i < 2^{-l_{i+j}}$ – Widerspruch zu (3.6).

Beliebiges Codealphabet mit b Elementen: In (3.5) $2^{-l_{i+1}}$ durch $b^{-l_{i+1}}$ ersetzen und die Werte in b-adischer Darstellung aufschreiben. Damit ist die Rückrichtung von Satz 3.1 bewiesen.

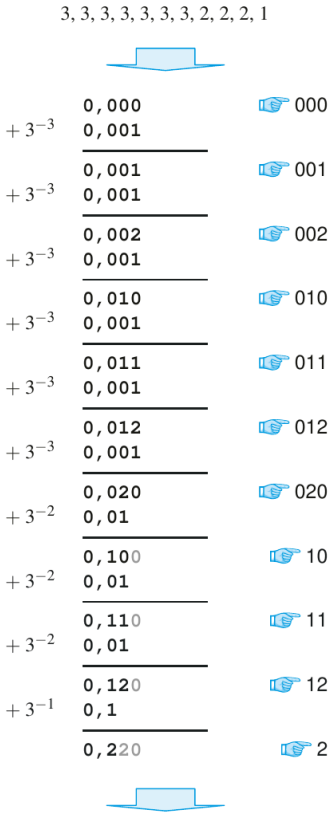


Abbildung 3.11: Die durchgeführte Rechnung bringt einen präfixfreien Code mit einem dreielementigen Codealphabet hervor.

Beispiel $b = 3$, Längen 3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 1 ($7 \cdot 3^{-3} + 3 \cdot 3^{-2} + 3^{-1} = \frac{25}{27} \leq 1$):
$C = \{000, 001, 002, 010, 011, 012, 020, 10, 11, 12, 2\}$.

3.4 Blockcodes

Ein **Blockcode** bzw. eine Blockcodierung liegt vor, wenn alle Codewörter die gleiche Länge haben.

Genetischer Code

Quellenalphabet und Codealphabet:

$$\Sigma := \{A, R, N, D, C, Q, E, G, H, I, L, K, M, F, P, S, T, W, Y, V\} \tag{3.7}$$

$$\Pi := \{A, G, C, T\}$$

Jedes Quellensymbol wird auf eine Sequenz von drei Codesymbolen abgebildet, ein **Triplett**. Erfunden hat diesen Code die Natur: der **genetische Code**. Die Quellensymbole sind die Abkürzungen der 20 **Aminosäuren**, aus denen in den Ribosomen Proteine gebildet werden; die Codesymbole sind die vier Basen **Guanin** (G), **Cytosin** (C), **Adenin** (A) und **Thymin** (T).

Tabelle 3.2: Der genetische Code

u	c(u)	Alternativen c(u)	u	c(u)	Alternativen c(u)
A	GCT	GCC, GCA, GCG	K	AAA	AAG
R	CGT	CGC, CGA, CGG, AGA, AGG	M	ATG	–
N	AAT	AAC	F	TTT	TTC
D	GAT	GAC	P	CCT	CCC, CCA, CCG
C	TGT	TGC	S	TCT	TCC, TCA, TCG, AGT, AGC
Q	CAA	CAG	T	ACT	ACC, ACA, ACG
E	GAA	GAG	W	TGG	–
G	GGT	GGC, GGA, GGG	Y	TAT	TAC
H	CAT	CAC	V	GTT	GTC, GTA, GTG
I	ATT	ATC, ATA			
L	TTA	TTG, CTT, CTC, CTA, CTG			

Die Zuordnung ist nicht eindeutig: Die meisten Aminosäuren können durch verschiedene Basentripletts codiert werden, besonders Arginin (R), Leucin (L) und Serin (S). Diese Mehrdeutigkeit ist ein Mittel zur Fehlerkorrektur und schützt vor Mutationen: Wird z. B. die dritte Base in GCT beschädigt, entsteht weiterhin Alanin (A).



Der Text (S. 179) spricht bei R, L und S von „jeweils fünf verschiedenen Codierungsmöglichkeiten“. Laut Tabelle 3.2 sind es je sechs – ein Codewort und fünf Alternativen; „fünf“ meint offenbar die Alternativen.

3.4.1 Zeichencodes

Brailleschrift

In der **Brailleschrift** (Louis Braille, frühes 19. Jahrhundert, für blinde Menschen) wird jeder Buchstabe durch eine Punktmatrix mit drei Zeilen und zwei Spalten dargestellt, die als Erhebung ins Papier gepresst wird. Sie ist äquivalent zu einem binären Blockcode der Länge 6: Die zweidimensionalen Muster werden Punkt für Punkt in eine 6-stellige Binärzahl übertragen.

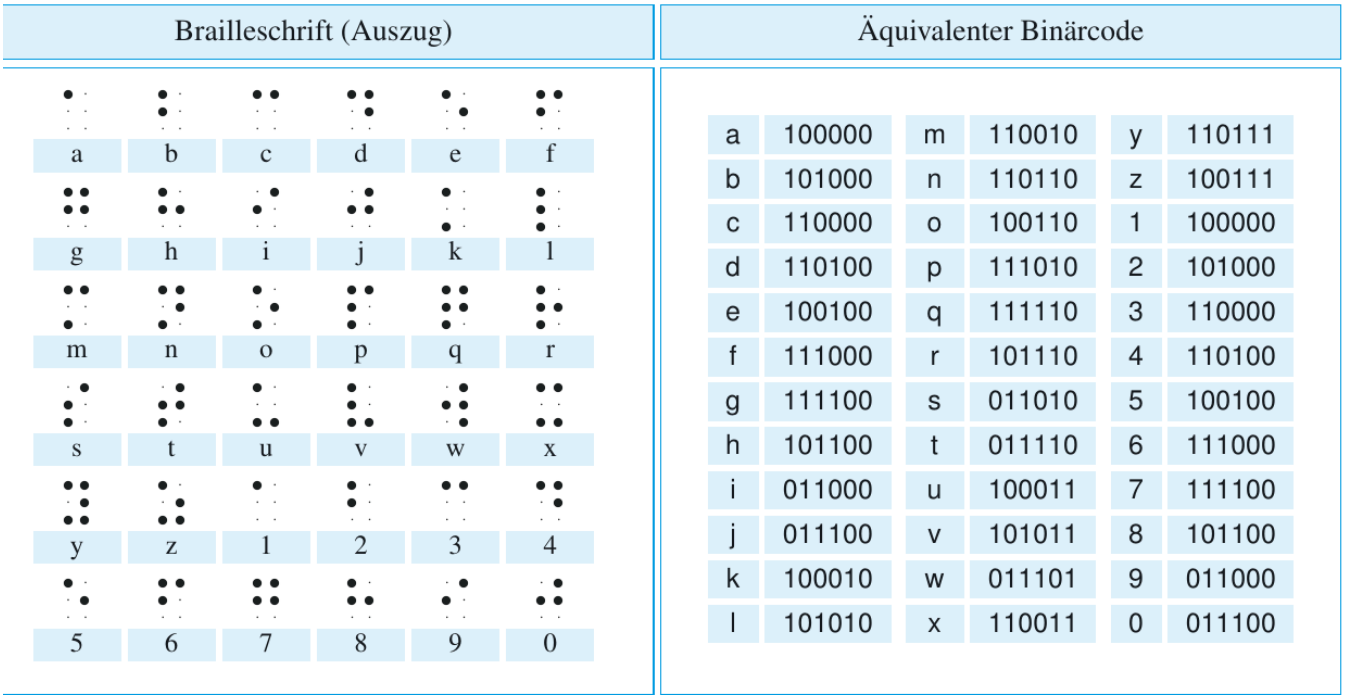


Abbildung 3.14: Die Brailleschrift ist ein klassischer Blockcode. Alle Codewörter haben die gleiche Länge.

ASCII

Der **American Standard Code for Information Interchange** (ASCII) besteht aus 128 Zeichen, die mit jeweils 7 Bit codiert werden (Tabelle 3.3). Neben den Schriftzeichen enthält er Steuerbefehle (u. a. zur Formatierung der Ausgabe auf textbasierten Terminals), die heute kaum noch eine Rolle spielen. Die ursprüngliche Form von 1963 hatte nur 99 Einträge (Steuerzeichen, Ziffern, Großbuchstaben, keine Kleinbuchstaben); 1974 wurde der Code als ISO 646 standardisiert.

Tabelle 3.3: Der American Standard Code for Information Interchange (ASCII). Zeile = erste Hexadezimalziffer, Spalte = zweite.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

- Im Hauptspeicher werden ASCII-Texte byteweise abgelegt; das freie 8. Bit wurde auf älteren Systemen manchmal als **Paritätsbit** verwendet: Es ergänzt das Bitmuster so, dass die Anzahl der Einsen gerade wird.
- Moderne Systeme füllen das Bitmuster mit `0` auf. So lässt sich der ASCII-Zeichensatz auf einen 8-Bit-Code erweitern, ohne die Kompatibilität zu verlieren; fast alle erweiterten Zeichensätze stimmen in den unteren 128 Zeichen mit ASCII überein.

ISO 8859

Die Teilnormen der ISO 8859 sind 8-Bit-Zeichensätze für verschiedene Sprachräume (Abb. 3.16):

Abbildung 3.16: Die Teilnormen der ISO 8859

x	ISO 8859-x	x	ISO 8859-x
1	Westeuropäisch (Latin-1)	9	Türkisch (Latin-5)
2	Osteuropäisch (Latin-2)	10	Nordisch (Latin-6)
3	Südeuropäisch (Latin-3)	11	Thai
4	Baltisch (Latin-4)	12	noch nicht verabschiedet
5	Kyrillisch	13	Baltisch (Latin-7)
6	Arabisch	14	Keltisch (Latin-8)
7	Griechisch	15	Westeuropäisch (Latin-9)
8	Hebräisch	16	Südosteuropäisch (Latin-10)

- **ISO 8859-1 (Latin-1)**: für den westeuropäischen Raum; enthält französische und skandinavische Sonderzeichen, die deutschen Umlaute und ß. Deutsche Texte lassen sich auch mit allen anderen Latin-Zeichensätzen außer Latin-7 verfassen.
- Problem der geringen Bitbreite von 8 Bit: Es gibt z. B. Tabellen für Deutsch und für Russisch, aber keine, die beide Schriftzeichen enthält; asiatische Sprachen sprengen das 255-Zeichen-Repertoire. Deshalb wurden ab Mitte der 80er Jahre Codierungen mit mehr als 8 Bit pro Zeichen entwickelt, die wichtigste ist der Unicode.

Unicode

Der **Unicode** ist eine universelle Symboltabelle, die jedem bekannten Zeichen ein eindeutiges Codewort zuordnet – auf jeder Hardware, unter jedem Betriebssystem und in jeder Programmiersprache dasselbe.

- Der Coderaum umfasst 17 Bereiche (**planes**) mit je 65.536 Zeichen, insgesamt mehr als eine Million Codewörter; bisher ist knapp 10 % ein Zeichen zugewiesen.
- **Plane 0**, die **Basic Multilingual Plane** (BMP), enthält die wichtigsten Zeichen. Ihre ersten 256 Zeichen sind identisch mit Latin-1 (ISO 8859-1) – Rückwärtskompatibilität.
- Jedes Zeichen der BMP lässt sich mit 16 Bit codieren (z. B. Java: 2 Byte pro Zeichen im Hauptspeicher).

Auf Datenträgern werden Unicode-Texte im **Universal Transformation Format** (UTF) gespeichert, einer Formatfamilie mit UTF-8, UTF-16 und UTF-32. Das Unicode-Bitmuster hat 21 Bit (`zzzzz yyyyyyyy xxxxxxxx`):

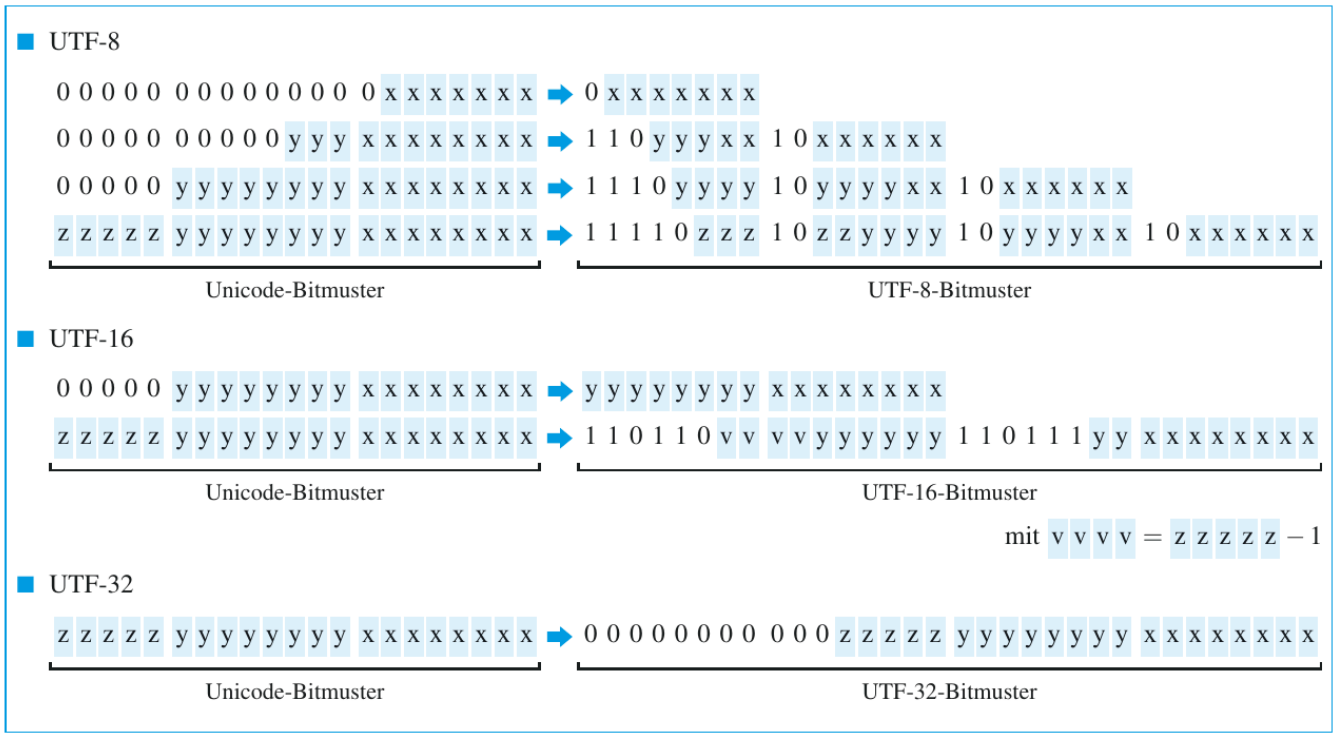


Abbildung 3.18: Konvertierung von Unicode-Zeichen in das UTF-Format

Format	Eigenschaften
UTF-8	Besonders für Texte mit vielen ASCII-Zeichen. Jedes ASCII-Zeichen wird mit dem gleichen Bitmuster codiert → vollständig rückwärtskompatibel. Zeichen oberhalb der 128-Zeichen-Grenze werden mit 16, 24 oder 32 Bit dargestellt. Ein verlängertes Codewort erkennt man an den führenden Einsen; ihre Anzahl ist die Anzahl der Bytes des Zeichens. Jedes Folgebyte beginnt mit `10`, sodass Anfang und Ende einer Zeichensequenz auch dann sicher erkannt werden, wenn erst mitten in einer laufenden Übertragung mitgelesen wird.
UTF-16	Für Texte mit wenigen ASCII-, aber vielen BMP-Zeichen. Die ersten 65.536 Zeichen (die BMP) werden mit zwei Byte codiert, höhere Zeichen mit 32 Bit: die ersten 16 heißen **High-Surrogate**, die folgenden 16 **Low-Surrogate**; beide beginnen zur Unterscheidung mit einer eigenen Bitsequenz. Die ersten 5 Unicode-Bits werden durch Subtraktion von eins auf 4 Bit komprimiert; das geht bedenkenlos, weil der einzige kritische Fall $zzzzz = 00000$ schon durch die 16-Bit-Codierung abgedeckt ist.
UTF-32	Jedes Zeichen ausnahmslos mit 4 Byte. Ein ASCII-Text braucht viermal so viel Platz wie in UTF-8, ein Text aus BMP-Zeichen doppelt so viel wie in UTF-16. Vorteil: einfache Handhabung – die Position des n-ten Zeichens ist direkt berechenbar, und aus der Dateigröße ergibt sich sofort die Anzahl der Zeichen.

3.4.2 Zahlencodes

Tetraden-Codes codieren Dezimalzahlen ziffernweise; jede Ziffer wird durch eine vier Bit breite **Tetrade** dargestellt. Da nur 10 der 16 Bitmuster gebraucht werden, gibt es in jedem Tetraden-Code 6 **Pseudo-Tetraden** ohne Bedeutung.

Tabelle 3.4: Tetraden-Codes in der Übersicht

Ziffer	BCD-Code	Stibitz-Code	Aiken-Code	Gray-Code
0	0000	0011	0000	0000
1	0001	0100	0001	0001
2	0010	0101	0010	0011
3	0011	0110	0011	0010
4	0100	0111	0100	0110
5	0101	1000	1011	0111
6	0110	1001	1100	0101
7	0111	1010	1101	0100
8	1000	1011	1110	1100
9	1001	1100	1111	1101

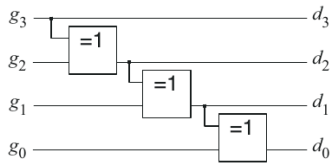
Code	Eigenschaften
BCD-Code	Am häufigsten eingesetzt. Die Bits einer Tetrade haben die Wertigkeiten 8, 4, 2 und 1 (**8421-Code**); jede Tetrade ist eine gewöhnliche Binärzahl. Codewort einer Ziffer = Binärdarstellung, mit führenden Nullen auf vier Bit ergänzt.
Stibitz-Code	Bitmuster gegenüber BCD um drei verschoben (**Excess-3-Code**). Das **Neunerkomplement** $9 - z$ einer Ziffer z entsteht durch Invertieren aller Bits; das Neunerkomplement wird für die Subtraktion gebraucht, arithmetische Operationen sind daher einfacher als im BCD-Code.
Aiken-Code	Neunerkomplement ebenfalls durch Invertieren aller Bits; zusätzlich hat jede Bitposition wieder eine feste Stelligkeit 2, 4, 2, 1 (**2421-Code**).
Gray-Code	**Progressiver** oder **einschrittiger Code**: Die Bitmuster zweier benachbarter Ziffern unterscheiden sich in genau einem Bit. Beim Wechsel von 9 auf 0 geht die Eigenschaft verloren. Abhilfe: der **erweiterte Gray-Code** (Tabelle 3.5), der alle Zahlen von 0 bis 15 in einer Tetrade codiert; beim Überlauf wechselt `1000` durch Änderung eines Bits zu `0000`.

Tabelle 3.5: Der erweiterte Gray-Code entsteht aus dem Gray-Code durch die Hinzunahme von 6 weiteren Bitmustern.

10	11	12	13	14	15
1111	1110	1010	1011	1001	1000

Rechnen mit progressiven Codes ist schwierig; deshalb übersetzt man den Gray-Code vor arithmetischen Operationen in den Binärcode – per Programm oder mit einfachen Hardwareschaltungen:

■ Parallele Konvertierung



■ Serielle Konvertierung

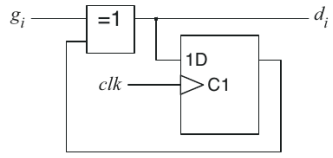


Abbildung 3.19: Hardwareschaltungen für die Konvertierung des Gray-Codes in das Binärsystem

Parallel: $d_3 = g_3$, jedes weitere Binärbit ist das XOR (=1) des nächsthöheren Binärbits mit dem Gray-Bit ($d_i = d_{i+1} \oplus g_i$). Seriell: ein XOR-Gatter mit rückgekoppeltem D-Flipflop.

3.4.3 Lineare Codes

Ist das Codealphabet Π ein endlicher Körper $\mathbb{F}$, lässt sich jedes Codewort als Element des Vektorraums $\mathbb{F}^n$ auffassen, wobei n die Länge der Codewörter ist.

Abbildung 3.20: Alle Codewörter sind Elemente des Vektorraums $\mathbb{Z}_2^3$.

$\boldsymbol{u}$	$c_1(\boldsymbol{u})$	$c_2(\boldsymbol{u})$	$c_3(\boldsymbol{u})$	$c_4(\boldsymbol{u})$
00	000	000	110	000
01	011	101	101	001
10	101	011	011	010
11	110	110	000	100

- c_1, c_2, c_3 sind verschiedene Codierungen (sie bilden die Quellensymbole unterschiedlich ab), erzeugen aber den gleichen Code – die Mengen ihrer Codewörter sind identisch. Man muss also zuweilen pedantisch zwischen Codierung und Code unterscheiden.
- Dieser Code ist nicht nur Teilmenge von $\mathbb{Z}_2^3$, sondern ein Untervektorraum: Er ist **linear**.

Definition 3.4 (Linearer Code)

Sei $\mathbb{F}^n$ ein Vektorraum über einem endlichen Körper $\mathbb{F}$. Die Menge

$$C \subseteq \mathbb{F}^n \text{ heißt } \textit{linearer } [n,k]\textit{-Code},$$

wenn sie ein k-dimensionaler Untervektorraum von $\mathbb{F}^n$ ist.

Lineare Codierungen sind Codierungen, die einen linearen Code erzeugen: c_1, c_2, c_3 sind linear, c_4 nicht.

Paritätscode

c_1, c_2, c_3 erzeugen den geraden **Paritätscode** (*parity code*): Alle Codewörter haben eine gerade Anzahl an Einsen.

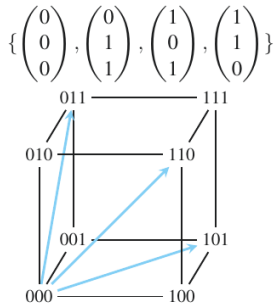


Abbildung 3.21: Der Paritätscode, dargestellt als Untervektorraum von $\mathbb{Z}_2^3$

Generatormatrix: Für einen linearen Code mit Codewortlänge n und Basis $\{\boldsymbol{b}_1, \ldots, \boldsymbol{b}_k\}$ werden die Basisvektoren zeilenweise zu einer Matrix G mit k Zeilen und n Spalten aufgeschrieben. Die Codewörter entstehen, indem die Vektoren aus $\mathbb{F}^k$ von links mit der Matrix multipliziert werden:

$$C = \{(u_0 \ \ldots \ u_{k-1}) \cdot G \mid (u_0 \ \ldots \ u_{k-1}) \in \mathbb{F}^k\}$$

Der Paritätscode aus Abbildung 3.21 wird von den Basisvektoren $(1,0,1)^T$ und $(0,1,1)^T$ aufgespannt:

$$G_1 := \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \tag{3.8}$$

Eine Generatormatrix beschreibt nicht nur einen Code, sondern auch eine Codierung:

$$(u_0 \ \ldots \ u_{k-1}) \mapsto (u_0 \ \ldots \ u_{k-1}) \cdot G_1 \tag{3.9}$$

Beispiel: Die Generatormatrizen G_1 (3.8) und G_2 (3.10) erzeugen die Codierungen c_1 und c_2 (Abb. 3.22)

G_1: $(0\,0) \cdot G_1 = (0\,0\,0), (0\,1) \cdot G_1 = (0\,1\,1), (1\,0) \cdot G_1 = (1\,0\,1), (1\,1) \cdot G_1 = (1\,1\,0) \to c_1$.

c_2 entsteht aus der Matrix (3.8) durch Vertauschen der ersten beiden Spalten:

$$G_2 := \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \tag{3.10}$$

$(0\,0) \cdot G_2 = (0\,0\,0), (0\,1) \cdot G_2 = (1\,0\,1), (1\,0) \cdot G_2 = (0\,1\,1), (1\,1) \cdot G_2 = (1\,1\,0) \to c_2$.

Für c_3 gibt es keine Generatormatrix, obwohl auch sie linear ist: Der Nullvektor wird von jeder Matrix auf den Nullvektor abgebildet, eine Generatormatrix kann es also nur geben, wenn die Nachricht `0…0` auf das Codewort `0…0` abgebildet wird; c_3 bildet `00` auf `110` ab.

c_1 ist die wichtigste der drei: Die Codewörter entstehen durch Anhängen eines Prüfbits, des **Paritätsbits**. Eine Codierung, die eine Nachricht lediglich verlängert, heißt systematisch:

Definition 3.5 (Systematische Codierung)

Eine Codierung heißt *systematisch*, wenn die codierte Nachricht die Originalnachricht als Anfangsstück enthält.

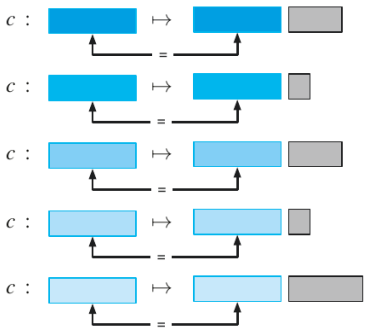


Abbildung 3.23: Systematische Codierungen sind so beschaffen, dass die Originalnachricht lediglich verlängert wird, aber ansonsten unverändert bleibt.

Paritätscode der Länge $n+1$: Die Generatormatrix entsteht, indem die $n \times n$-Einheitsmatrix um eine mit Einsen gefüllte Spalte ergänzt wird. Die Einheitsmatrix sorgt dafür, dass der Anfang des Codeworts mit $\boldsymbol{u}$ übereinstimmt; das letzte Bit ist die Summe der anderen Bits modulo 2, also genau dann 1, wenn die Anzahl der Einsen in $\boldsymbol{u}$ ungerade ist.

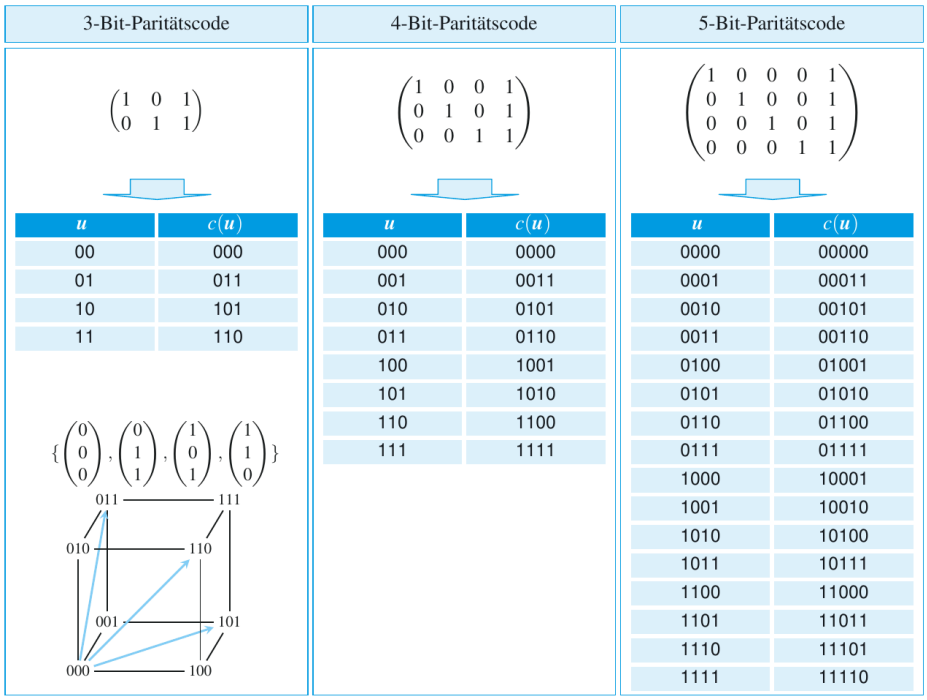


Abbildung 3.24: Der gerade Paritätscode ist ein linearer Blockcode. In allen Codewörtern ist die Anzahl der Einsen gerade.

Der Paritätscode hat eine weitere Eigenschaft: Rotiert man ein Codewort (Teilstück an einer Seite entfernen und an der anderen wieder anhängen), entsteht erneut ein Codewort.

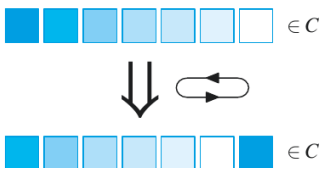


Abbildung 3.25: Zyklische Codes besitzen die Eigenschaft, dass die zyklische Verschiebung (Rotation) eines Codeworts wieder zu einem Codewort führt.

Definition 3.6 (Zyklischer Code)

Ein Code heißt *zyklisch*, wenn jede zyklische Verschiebung eines Codeworts ebenfalls ein Codewort ist.

Hamming-Code

Der Untervektorraum von $\mathbb{Z}_2^7$, der von den Basisvektoren $(1,1,1,0,0,0,0)^T$, $(1,0,0,1,1,0,0)^T$, $(0,1,0,1,0,1,0)^T$, $(1,1,0,1,0,0,1)^T$ aufgespannt wird (Abb. 3.26), hat die Generatormatrix

$$G := \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \tag{3.11}$$

Sie erzeugt den $[7,4]$-**Hamming-Code** (Codewörter in Abbildung 3.27). Hamming-Codes spielen in der Kanalcodierung eine große Rolle (ausführlich in 6.4.2).

Dualer Code: Zu jedem Untervektorraum $U \subseteq V$ existiert ein Untervektorraum $U^\perp \subseteq V$, dessen Vektoren orthogonal auf den Vektoren von U stehen (2.5.2). Seine Elemente bilden ebenfalls einen linearen Code.

Definition 3.7 (Dualer Code)

> Definiert der Untervektorraum U den linearen Code C, dann definiert der Orthogonalraum $U^\perp$ den zu C *dualen Code*.

Generatormatrix von $U^\perp$:

1. Die Generatormatrix von U in die systematische Form bringen (Einheitsmatrix links).
2. Die Spalten rechts der Einheitsmatrix als Zeilenvektoren aufschreiben.
3. Diese Teilmatrix rechts um die Einheitsmatrix ergänzen.

Für die Matrix (3.11) ergibt sich (Rechnung auf S. 151) die Generatormatrix des **Simplex-Codes** (mehr dazu in 6.4.7):

$$G^\perp = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \tag{3.12}$$

Erweiterter Hamming-Code

$$G' := \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

G' unterscheidet sich von der **Hamming-Matrix** (3.11) nur durch eine zusätzliche Spalte: Der **erweiterte Hamming-Code** hat genauso viele Elemente wie der $[7,4]$-Hamming-Code, aber um 1 Bit verlängerte Codewörter. Das zusätzliche Bit ist ein Paritätsbit; es ist genau dann 1, wenn die Anzahl der Einsen in $\boldsymbol{u}$ ungerade ist.

Hamming-Code

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

u	$c(u)$
0000	0000000
0001	1101001
0010	0101010
0011	1000011
0100	1001100
0101	0100101
0110	1100110
0111	0001111
1000	1110000
1001	0011001
1010	1011010
1011	0110011
1100	0111100
1101	1010101
1110	0010110
1111	1111111

Simplex-Code

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

u	$c(u)$
000	0000000
001	1101001
010	1011010
011	0110011
100	0111100
101	1010101
110	1100110
111	0001111

Erweiterter Hamming-Code

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

u	$c(u)$
0000	00000000
0001	11010010
0010	01010101
0011	10000111
0100	10011001
0101	01001011
0110	11001100
0111	00011110
1000	11100001
1001	00110011
1010	10110100
1011	01100110
1100	01111000
1101	10101010
1110	00101101
1111	11111111

Abbildung 3.27: Der Hamming-Code, der Simplex-Code und der erweiterte Hamming-Code im Vergleich

Der duale Code des erweiterten Hamming-Codes (Abb. 3.28): Die Generatormatrix G' wird in die systematische Form gebracht, daraus die Generatormatrix des Orthogonalraums abgeleitet und diese wieder in die systematische Form gebracht:

$$G' \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

(systematische Form von G' → Orthogonalraum → dessen systematische Form). Beide systematischen Matrizen sind identisch, d. h. beide Untervektorräume sind gleich: Der erweiterte Hamming-Code ist mit seinem dualen Code identisch, er ist **selbstdual**.

Definition 3.8 (Selbstdualer Code)

Ein linearer Code C mit $C = C^{\perp}$ heißt *selbstdual*.

Äquivalente Codes

Die Generatormatrix

$$G'' := \begin{pmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{3.13}$$

erzeugt einen Code, der dem erweiterten Hamming-Code sehr ähnlich ist: Die Codewörter stehen in anderen Zeilen der Tabelle, und das Paritätsbit steht einmal an der ersten, einmal an der letzten Stelle. G'' entsteht aus G' durch elementare Zeilen- und Spaltenoperationen: Vertauschen bzw. Addieren von Zeilen ändert die Reihenfolge der Codewörter in der Tabelle, Vertauschen von Spalten die Reihenfolge der Bits in den Codewörtern. Solche Codes heißen **äquivalent**.

Definition 3.9 (Code-Äquivalenz)

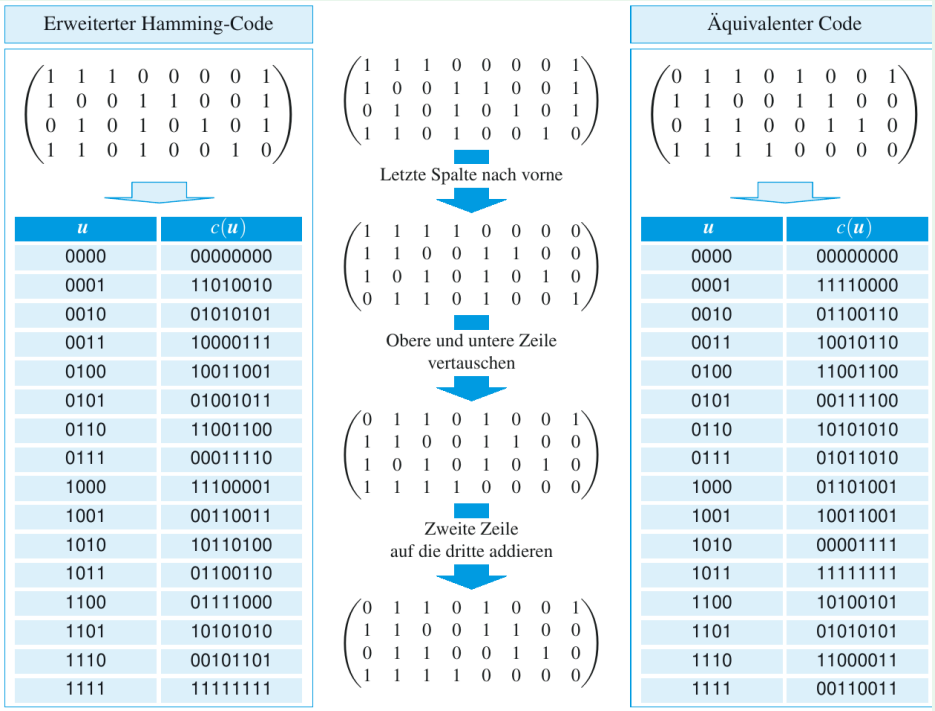
Zwei lineare Codes, die durch die Matrizen G_1 bzw. G_2 erzeugt werden, heißen *äquivalent*, wenn G_1 durch

- die Vertauschung von Zeilen und Spalten und
- die Addition von Zeilen

nach G_2 überführt werden kann.

Beispiel: Von G' nach G'' (Abb. 3.29)

1. Letzte Spalte nach vorne.
2. Obere und untere Zeile vertauschen.
3. Zweite Zeile auf die dritte addieren.



u	$c(u)$
0000	00000000
0001	11010010
0010	01010101
0011	10000111
0100	10011001
0101	01001011
0110	11001100
0111	00011110
1000	11100001
1001	00110011
1010	10110100
1011	01100110
1100	01111000
1101	10101010
1110	00101101
1111	11111111

u	$c(u)$
0000	00000000
0001	11110000
0010	01100110
0011	10010110
0100	11001100
0101	00111100
0110	10101010
0111	01011010
1000	01101001
1001	10011001
1010	00001111
1011	11111111
1100	10100101
1101	01010101
1110	11000011
1111	00110011

Abbildung 3.29: Der erweiterte Hamming-Code und der rechts abgedruckte Code sind äquivalent. Die Generatormatrizen lassen sich durch elementare Zeilen- und Spaltenumformungen ineinander überführen.

Zyklischer Hamming-Code

Zum Hamming-Code gibt es äquivalente Codes, die zyklisch sind. Konstruktion durch Umsortieren:

1. Zeilen von (3.11) in die neue Reihenfolge bringen:

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix} \tag{3.14}$$

2. Die Spalten, die genau eine 1 enthalten, nach vorne bringen → mittlere Matrix in Abbildung 3.30.

Rotiert man die Bits eines Codeworts dieses Codes um eine beliebige Anzahl Positionen nach links oder rechts, entsteht wieder ein Codewort: Der Hamming-Code ist zu einem zyklischen Code äquivalent, dem **zyklischen Hamming-Code**. Er ist nicht eindeutig: Spiegelt man die Codewörter eines zyklischen Codes, erhält man wieder einen zyklischen Code (rechte

Spalte). Generatormatrix dazu: Spalten in umgekehrter Reihenfolge aufschreiben und durch elementare Zeilenumformungen in die normalisierte Form bringen.

Hamming-Code

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

u	$c(u)$
0000	0000000
0001	1101001
0010	0101010
0011	1000011
0100	1001100
0101	0100101
0110	1100110
0111	0001111
1000	1110000
1001	0011001
1010	1011010
1011	0110011
1100	0111100
1101	1010101
1110	0010110
1111	1111111

Zyklischer Hamming-Code (1)

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

u	$c(u)$
0000	0000000
0001	0001101
0010	0010111
0011	0011010
0100	0100011
0101	0101110
0110	0110100
0111	0111001
1000	1000110
1001	1001011
1010	1010001
1011	1011100
1100	1100101
1101	1101000
1110	1110010
1111	1111111

Zyklischer Hamming-Code (2)

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

u	$c(u)$
0000	0000000
0001	0001011
0010	0010110
0011	0011101
0100	0100111
0101	0101100
0110	0110001
0111	0111010
1000	1000101
1001	1001110
1010	1010011
1011	1011000
1100	1100010
1101	1101001
1110	1110100
1111	1111111

Abbildung 3.30: Der mittlere und der rechte Code sind systematisch und zyklisch. Sie sind zu dem $[7, 4]$-Hamming-Code äquivalent.

3.5 Der Übertragungskanal

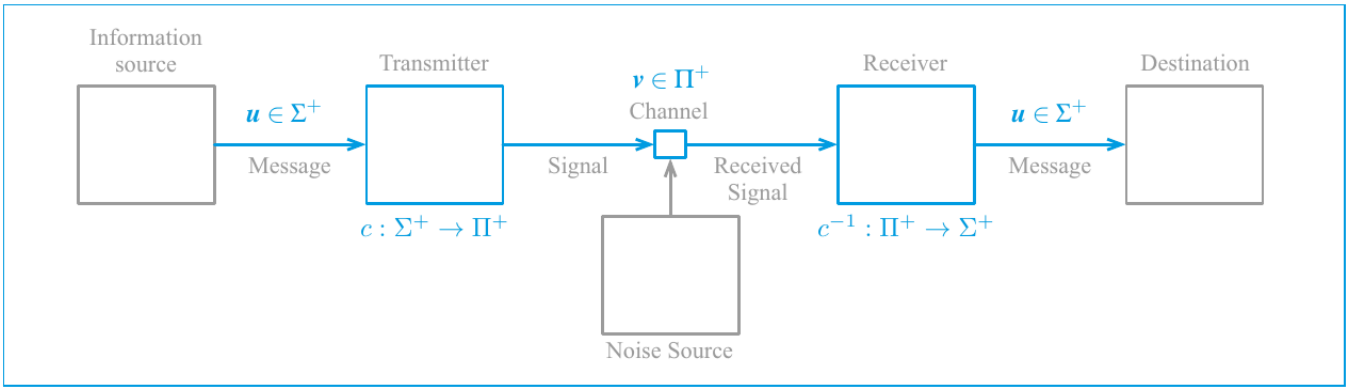


Abbildung 3.31: Unser Begriffsgerüst im Lichte des Shannon'schen Kommunikationsmodells

Die eingeführten Begriffe im Shannon'schen Übertragungsmodell:

- Dem Sender (*transmitter*) wird eine Nachricht $\boldsymbol{u} \in \Sigma^+$ übergeben, d. h. eine Sequenz von Zeichen aus dem Quellenalphabet Σ.
- Sie wird in ein Signal übersetzt, eine Reihe von Zeichen aus dem Codealphabet, und in den Übertragungskanal (*communication channel*) eingespeist. Zu jedem Zeitpunkt hat der Sender endlich viele Quellensymbole entgegengenommen und endlich viele Codesymbole erzeugt: $c(\boldsymbol{u}) = \boldsymbol{v}$ mit $\boldsymbol{u} \in \Sigma^+$, $\boldsymbol{v} \in \Pi^+$ und der Codierung c.
- Der Empfänger (*receiver*) nimmt das Signal entgegen und wandelt es in eine Nachricht zurück ($c^{-1} : \Pi^+ \to \Sigma^+$).

3.5.1 Kanalkapazität

Wie lässt sich die Kapazität des Übertragungskanals quantitativ erfassen (nach Shannons Arbeit)? Betrachtet wird ein Sender mit binärem Eingabestrom, der nach T Zeiteinheiten n Bits eingespeist hat; mit n Bits gibt es 2^n verschiedene Bitsequenzen.

Mit der Vereinbarung

$$N(T) := \text{Anzahl der erlaubten Signale der Länge } T$$

gilt: Ist $N(T) = 2^n$, so kann der Kanal in T Zeiteinheiten n Bits transportieren. Innerhalb von T Zeiteinheiten werden also durchschnittlich

$$\frac{\log_2 N(T)}{T} \tag{3.15}$$

Bits pro Zeiteinheit übertragen. Für immer größere T stabilisiert sich dieser Quotient bei allen praxisrelevanten Kanälen.

Definition 3.10 (Kanalkapazität nach Shannon, 1948)

Ein diskreter Übertragungskanal besitzt die *Kanalkapazität*

$$C := \lim_{T \to \infty} \frac{\log_2 N(T)}{T}$$

Die Kanalkapazität ist die maximale Übertragungsrate in Bits pro Zeiteinheit, die aufgrund der physikalischen Randbedingungen über lange Zeiträume aufrechterhalten werden kann.

3.5.2 Kapazität des Morse-Kanals

Zwei Besonderheiten der Morse-Übertragung:

- Die Codesymbole dauern unterschiedlich lange (Abb. 3.32): ‚·' = Leitung 1 Zeiteinheit geschlossen, 1 offen (2 Zeiteinheiten); ‚–' = 3 geschlossen, 1 offen (4 Zeiteinheiten); Buchstabenpause = 2 und Wortpause = 6 Zeiteinheiten offen.
- Nicht jede Sequenz ist ein wohlgeformtes Signal: Die Leitung darf nicht länger als drei Zeiteinheiten geschlossen sein, und Pausenzeichen dürfen nicht direkt hintereinander gesendet werden (sonst wären zwei Buchstabenpausen nicht von einer Wortpause zu unterscheiden).

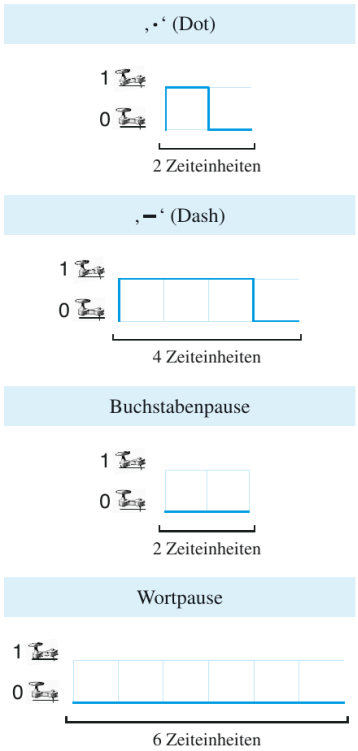


Abbildung 3.32: Physikalische Repräsentation der Morse-Codesymbole auf dem Übertragungskanal

$N(T)$ für immer größere T direkt abzuzählen stößt an praktische Grenzen, da die Anzahl der Signale exponentiell mit T wächst. Stattdessen systematisch:

Ohne die Pausen-Restriktion

Zustandsübergangsdiagramm mit einem Zustand und vier Kanten für ‚·‘, ‚–‘ und die beiden Pausen; die Kantenmarkierungen zeigen den Signalverlauf.

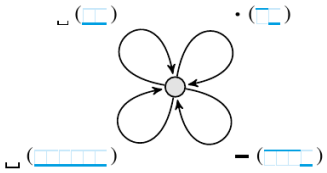


Abbildung 3.33: Modellierung des Morse-Übertragungskanals mithilfe eines Zustandsübergangsdiagramms

Ein langes Signal der Länge T endet auf eine von drei Arten:

- zuletzt ‚·‘ übertragen (endet mit 1 Zeiteinheit geschlossen, 1 offen): $N(T-2)$ solche Sequenzen;
- zuletzt ‚–‘ (3 geschlossen, 1 offen): $N(T-4)$;
- zuletzt eine Buchstaben- oder Wortpause (endet mit 2 offenen Zeiteinheiten): $N(T-2)$.

Daraus die **Rekurrenzgleichung**

$$N(T) = N(T-2) + N(T-4) + N(T-2)$$



Eigene Anmerkung: Die Wortpause erscheint nicht als eigener Summand $N(T-6)$, weil $N(T)$ Signale zählt: Ohne die Restriktion ist das Signal einer Wortpause (6 Zeiteinheiten offen) dasselbe wie das dreier Buchstabenpausen.

Allgemein, für Symbole der Dauer $T_1, \ldots, T_n$:

$$N(T) = N(T-T_1) + N(T-T_2) + \ldots + N(T-T_n) \tag{3.16}$$

Nach einem Ergebnis der Rekursionstheorie nähert sich die Lösung von (3.16) asymptotisch dem Wert

$$A \cdot {X_0}^T \tag{3.17}$$

mit einer Konstanten A und der größten reellwertigen Lösung X_0 der **charakteristischen Gleichung**

$$X^{-T_1} + X^{-T_2} + \ldots + X^{-T_n} = 1.$$

Eingesetzt in Definition 3.10:

$$C = \lim_{T\to\infty} \frac{\log_2 N(T)}{T} = \lim_{T\to\infty} \frac{\log_2 A \cdot {X_0}^T}{T} = \lim_{T\to\infty} \frac{\log_2 A + T\log_2 X_0}{T} = \lim_{T\to\infty} \frac{\log_2 A}{T} + \lim_{T\to\infty} \frac{T\log_2 X_0}{T} = \log_2 X_0$$

Beispiel: Morse-Kanal ohne Pausen-Restriktion

Die größte reellwertige Lösung von $X^{-2} + X^{-4} + X^{-2} = 1$ ist $X_0 \approx 1{,}5538$, also

$$C = \log_2 X_0 \approx \log_2 1{,}5538 \approx 0{,}6358 \ \frac{\text{Bits}}{\text{Zeiteinheit}}$$

Mit der Pausen-Restriktion

Dass Pausensymbole nicht direkt hintereinander gesendet werden dürfen, modelliert ein Zustandsdiagramm mit zwei Zuständen (Abb. 3.34). Es lässt sich nicht direkt in eine Rekurrenzgleichung übersetzen, aber in ein äquivalentes Diagramm mit nur einem Zustand reduzieren (Abb. 3.35): Kanten für ‚·‘ (2), ‚–‘ (4), ‚·‘ + Buchstabenpause (4), ‚–‘ + Buchstabenpause (6), ‚·‘ + Wortpause (8), ‚–‘ + Wortpause (10).

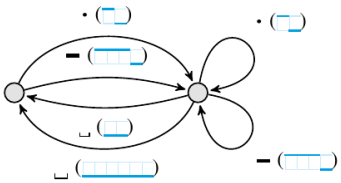


Abbildung 3.34: Das gezeigte Zustandsübergangsdiagramm berücksichtigt, dass Buchstaben- und Wortpausen nicht direkt hintereinander gesendet werden dürfen.

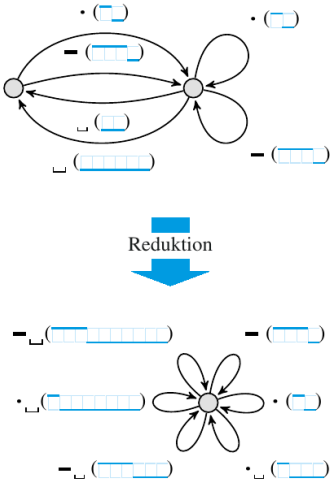


Abbildung 3.35: Reduktion auf ein Zustandsübergangsdiagramm mit einem einzigen Zustand

Beispiel: Morse-Kanal mit Pausen-Restriktion

Gesucht ist die größte reellwertige Lösung von

$$X^{-2} + X^{-4} + X^{-4} + X^{-6} + X^{-8} + X^{-10} = 1$$

Mit $X_0 \approx 1{,}49305$ ergibt sich eine geringfügig kleinere Kanalkapazität:

$$C = \log_2 X_0 \approx \log_2 1{,}49305 \approx 0{,}57826 \ \frac{\text{Bits}}{\text{Zeiteinheit}} \tag{3.18}$$

Der Morse-Übertragungskanal überträgt in einer Zeiteinheit bis zu 0,57826 Bits.

Übertragungsrate in Bits pro Sekunde

Geschwindigkeit eines Telegrafisten: Einheit **WpM** (*Wörter pro Minute*), Referenzwort PARIS = ‚· – – ·‘ ‚· –‘ ‚· – ·‘ ‚· ·‘ ‚· · ·‘, gesprochen „DiDaDaDit DiDah DiDaDit DiDit DiDiDit“. Mit Buchstabenpausen und abschließender Wortpause besteht es aus 50 Zeiteinheiten, ist also 50-mal so lang wie ein einzelnes „Di“ oder „Dit“: PARIS ist **50 Dits** lang.

Anzahl der Dits pro Minute und Dauer eines Dits sind umgekehrt proportional zueinander:

Tabelle 3.6: Typische Morse-Geschwindigkeiten im Vergleich

WpM	Dits/Minute	Länge eines Dits	
1	50	1200 msec	
5	250	240 msec	Amateurbereich
10	500	120 msec	
15	750	80 msec	
20	1000	60 msec	
25	1250	48 msec	Schnelltelegrafie
30	1500	40 msec	
35	1750	34,29 msec	
40	2000	30 msec	Profi-Telegrafie
45	2250	26,67 msec	
50	2500	24 msec	
55	2750	21,82 msec	
60	3000	20 msec	
65	3250	18,46 msec	
70	3500	17,14 msec	
75	3750	16 msec	
75,2	3760	15,96 msec	Weltrekord

Amateure übertragen weniger als 5 WpM, geübte Telegrafisten etwa 15 WpM, ab ca. 25 WpM beginnt die Schnelltelegrafie, Profis erreichen ca. 40 WpM. Weltrekord: 75,2 WpM (Theodore R. McElroy, 1939).

Beispiel: Kapazität des Morse-Kanals in Bits pro Sekunde

Beim Weltrekord dauert ein Dit 15,96 Millisekunden. Mit dieser Zeiteinheit folgt aus (3.18):

$$C = 0{,}57826 \, \frac{\text{Bits}}{\frac{15{,}96}{1000}\,\text{sec}} = 36{,}232 \, \frac{\text{Bits}}{\text{sec}} \tag{3.19}$$

Der Morse-Übertragungskanal hat eine Kapazität von ca. 36 Bits pro Sekunde.

3.6 Der Informationsbegriff

Bisher war von übertragenen Bits die Rede, aber nicht von übertragener **Information**. Leitgedanke nach Fred Attneave:

> *Information ist das, was Ungewissheit beseitigt oder reduziert.*

Orakelbefragung

Ein System ist mit einem 3 Bit langen Binärschlüssel gesichert; ein Orakel verrät auf Nachfrage nacheinander die drei Binärziffern. Zu Beginn besitzen wir 0 Bits an Information. Ist der Schlüssel zufällig gewählt, sind beide Antworten auf jede Frage gleich wahrscheinlich; jede Antwort entspricht 1 Bit an Information und halbiert die Anzahl der in Frage kommenden Sequenzen: 8 → 4 → 2 → 1.

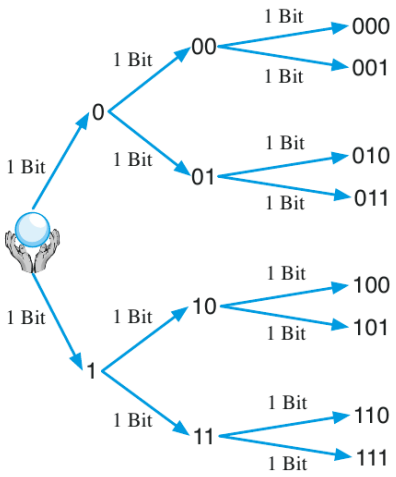


Abbildung 3.37: Die möglichen Ausgänge der Orakelbefragung

Jeder Pfad von der Wurzel zu einem Blatt beschreibt einen möglichen Verlauf; jede Kante ist mit der Information markiert, die beim Eintreten des Ereignisses gewonnen wird.

Zusammenhang zwischen Informationsgewinn und Unsicherheitsverlust:

- 8 Möglichkeiten auf 4 einzugrenzen entspricht einer Information von 1 Bit, auf 2 von 2 Bit, auf 1 von 3 Bit.
- Allgemein, für natürliche Zahlen $n \geq m$: 2^n Möglichkeiten auf 2^m Möglichkeiten einzugrenzen entspricht einer Information von $n - m$ Bit.
- Ohne Beschränkung auf Zweierpotenzen: n Möglichkeiten auf m Möglichkeiten einzugrenzen entspricht einer Information von $\log_2 n - \log_2 m$ Bit.

Merksatz

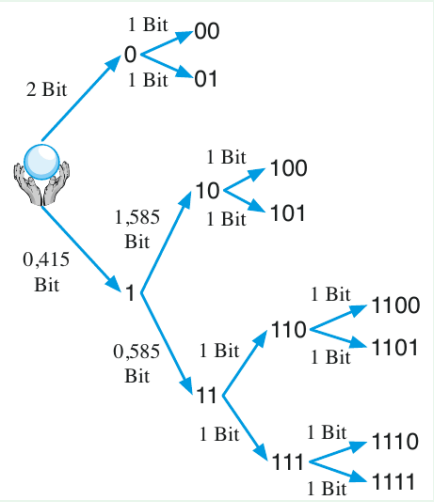
n Möglichkeiten auf m Möglichkeiten einzugrenzen entspricht einer Information von

$$\log_2 \frac{n}{m} \text{ Bit}$$

Beispiel: Variation der Orakelbefragung (Abb. 3.38, 3.39)

Der gesuchte Schlüssel stammt aus der Menge $\{00, 01, 100, 101, 1100, 1101, 1110, 1111\}$. Die gewonnene Information hängt jetzt von der Antwort ab:

- Erste Antwort „0“: 8 Möglichkeiten auf 2 eingegrenzt, $\log_2 \frac{8}{2}\,\text{Bit} = \log_2 4\,\text{Bit} = 2\,\text{Bit}$.
- Erste Antwort „1“: 6 Möglichkeiten bleiben, $\log_2 \frac{8}{6}\,\text{Bit} = \log_2 \frac{4}{3}\,\text{Bit} \approx 0{,}415\,\text{Bit}$.



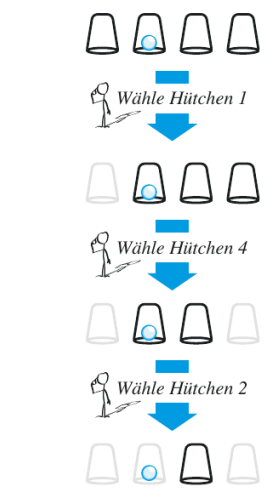
Die Summe der Einzelinformationen ergibt für jede Sequenz 3 Bit („8 Möglichkeiten auf 1 eingrenzen“):

Sequenz `00`, `01`	$I = (2 + 1)\,\text{Bit} = 3\,\text{Bit}$
Sequenz `100`, `101`	$I = (0{,}415 + 1{,}585 + 1)\,\text{Bit} = 3\,\text{Bit}$
Sequenz `1100`, `1101`, `1110`, `1111`	$I = (0{,}415 + 0{,}585 + 1 + 1)\,\text{Bit} = 3\,\text{Bit}$

Beispiel: Hütchenspiel (Abb. 3.40)

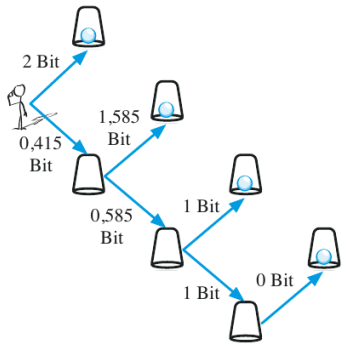
Beim **Hütchenspiel** (*shell game*) wird eine Perle unter einem von vier Hütchen versteckt; gesucht ist das Hütchen mit der Perle. Ohne Anhaltspunkt wird zufällig geraten, bei Fehlversuch unter den verbleibenden weiter.

- Möglicher Verlauf des Hütchenspiels



- Es gibt vier mögliche Spielverläufe

  1) *Erfolg im 1. Rateversuch*
  2) *Erfolg im 2. Rateversuch*
  3) *Erfolg im 3. Rateversuch*
  4) *Erfolg im 4. Rateversuch*



- Richtig im 1. Versuch: $4 \to 1$, $\log_2 \frac{4}{1}\,\text{Bit} = 2\,\text{Bit}$; falsch: $4 \to 3$, $\log_2 \frac{4}{3}\,\text{Bit} \approx 0{,}415\,\text{Bit}$.
- Richtig im 2. Versuch: $3 \to 1$, $\log_2 \frac{3}{1}\,\text{Bit} \approx 1{,}585\,\text{Bit}$; falsch: $3 \to 2$, $\log_2 \frac{3}{2}\,\text{Bit} \approx 0{,}585\,\text{Bit}$.
- 3. Versuch: je 1 Bit. Der vierte Versuch bringt 0 Bit: Sind drei leere Hütchen aufgedeckt, ist klar, wo die Perle liegt.

Am Ende sind immer 4 Möglichkeiten auf eine reduziert, also 2 Bit gewonnen:

Erfolg im 1. Rateversuch	$I = 2\,\text{Bit}$
Erfolg im 2. Rateversuch	$I = (0{,}415 + 1{,}585)\,\text{Bit} = 2\,\text{Bit}$
Erfolg im 3. Rateversuch	$I = (0{,}415 + 0{,}585 + 1)\,\text{Bit} = 2\,\text{Bit}$
Erfolg im 4. Rateversuch	$I = (0{,}415 + 0{,}585 + 1)\,\text{Bit} = 2\,\text{Bit}$

Warum die Summe immer gleich ist: Auf dem Weg von der Wurzel zu einem Blatt mit k Kanten reduziert die 1. Antwort n Möglichkeiten auf m_1, die 2. Antwort m_1 auf m_2, ..., die k-te Antwort m_{k-1} auf 1. Kumulierter Informationsgehalt:

$$
\begin{aligned}
I &= \log_2 \frac{n}{m_1} + \log_2 \frac{m_1}{m_2} + \log_2 \frac{m_2}{m_3} + \ldots + \log_2 \frac{m_{k-1}}{1} \\
&= \log_2 n - \log_2 m_1 + \log_2 m_1 - \log_2 m_2 + \log_2 m_2 - \ldots - \log_2 m_{k-1} + \log_2 m_{k-1} - \log_2 1 \\
&= \log_2 n - \log_2 1 = \log_2 n
\end{aligned}
$$

Die Logarithmen löschen sich paarweise aus.

Informationsgehalt und Wahrscheinlichkeit

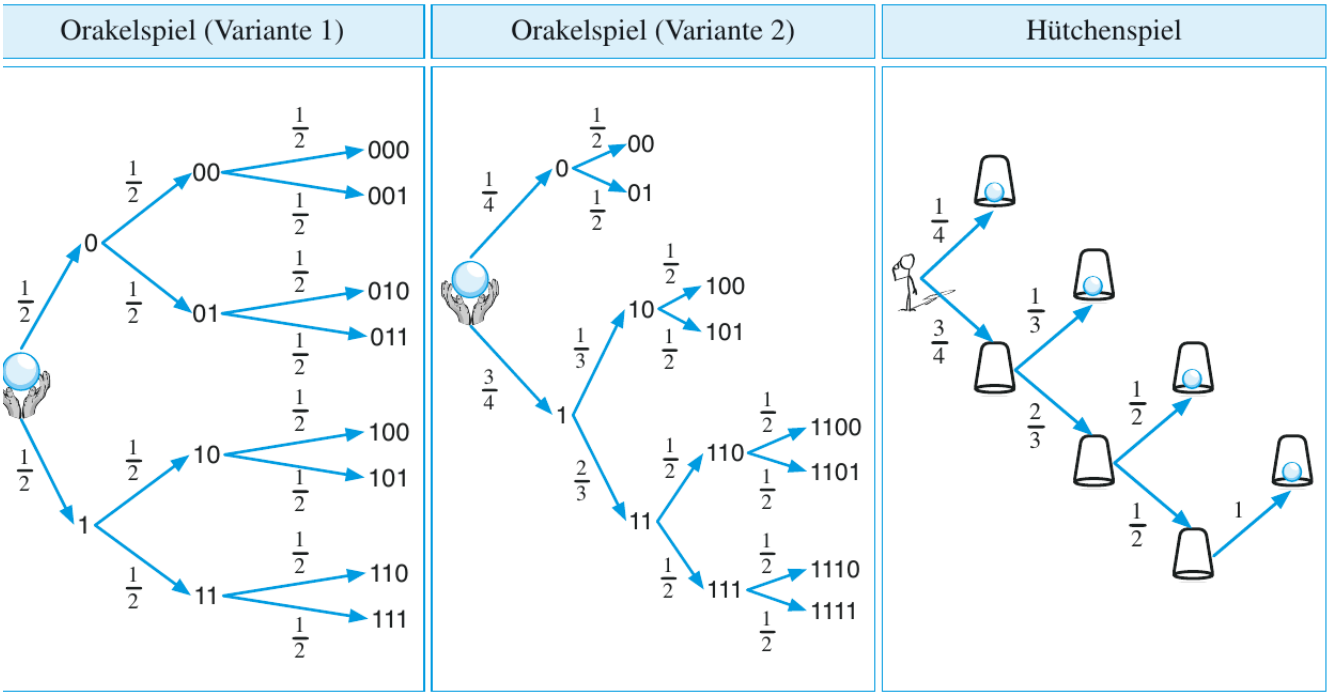


Abbildung 3.41: Die möglichen Spielverläufe haben verschiedene Wahrscheinlichkeiten. Ein vergleichender Blick mit den Originalbäumen macht deutlich, dass der Informationsgewinn in einem direkten Zusammenhang zu der Eintrittswahrscheinlichkeit eines Ereignisses steht.

Markiert man die Kanten statt mit der Information mit der Wahrscheinlichkeit des Ereignisses, zeigt sich: Ist p die Wahrscheinlichkeit, dass ein Ereignis eintritt, so liefert es exakt

$$\log_2 \frac{1}{p} = -\log_2 p \tag{3.20}$$

Bit an Information (z. B. $p = \frac{3}{4}$: $\log_2 \frac{4}{3} \approx 0{,}415$ Bit).

Definition 3.11 (Informationsgehalt)

Der Informationsgehalt eines Ereignisses, das mit der Wahrscheinlichkeit p ($p > 0$) eintritt, ist definiert als

$$I(p) := \log_2 \frac{1}{p} = -\log_2 p$$

Vier Eigenschaften des Informationsgehalts $I(p)$:

Eigenschaft	Formel	Bedeutung
nichtnegativ	$I(p) \geq 0$	p liegt in $(0;1]$, also $\frac{1}{p} \geq 1$ und $\log_2 \frac{1}{p} \geq 0$: Information reduziert Unsicherheit und ist nie negativ. $I(p) = 0$ genau dann, wenn $p = 1$ – das Ereignis steht ohnehin fest.
stetig	$p_i \to p \Rightarrow I(p_i) \to I(p)$	Eine kleine Änderung der Wahrscheinlichkeit führt nur zu einer kleinen Änderung der Information.
monoton	$p_1 < p_2 \Rightarrow I(p_1) > I(p_2)$	Je unwahrscheinlicher ein Ereignis, desto größer sein Informationsgehalt (häufige Füllwörter tragen kaum zur Information bei, seltene Begriffe oft den Inhalt eines Satzes).
additiv	$I(p_1 p_2) = I(p_1) + I(p_2)$	Für zwei stochastisch unabhängige Ereignisse mit den Wahrscheinlichkeiten p_1 und p_2 ist die Wahrscheinlichkeit des Gesamtereignisses $p = p_1 p_2$; seine Information ist die Summe der Einzelinformationen.

Additivität ausgerechnet:

$$I(p) = I(p_1 p_2) = \log_2 \frac{1}{p_1 p_2} = \log_2 \frac{1}{p_1} + \log_2 \frac{1}{p_2} = I(p_1) + I(p_2)$$

Alternativer, axiomatischer Weg (in vielen Lehrbüchern): Die Funktion $I(p)$ wird über die vier Eigenschaften Nichtnegativität, Stetigkeit, Monotonie und Additivität charakterisiert. Sie lassen nur $I(p) = \log \frac{1}{p}$ übrig; die Basis des Logarithmus ist dadurch nicht festgelegt. Die Wahl der Basis 2 (**Logarithmus dualis**) misst Information in der Einheit **Bit**.

Wiegeproblem

Von 8 Kugeln ① … ⑧ hat genau eine ein anderes Gewicht; man weiß weder welche, noch ob sie schwerer oder leichter ist. Der Abweichler soll mit einer Pendelwaage in möglichst wenigen Wiegeschritten identifiziert werden.

Ad-hoc-Strategie (Abb. 3.42): erst zwei Vierergruppen wiegen, dann die verbleibenden 4 Kugeln in zwei Zweiergruppen usw.; der Abweichler ist nach höchstens vier Wiegeschritten gefunden.

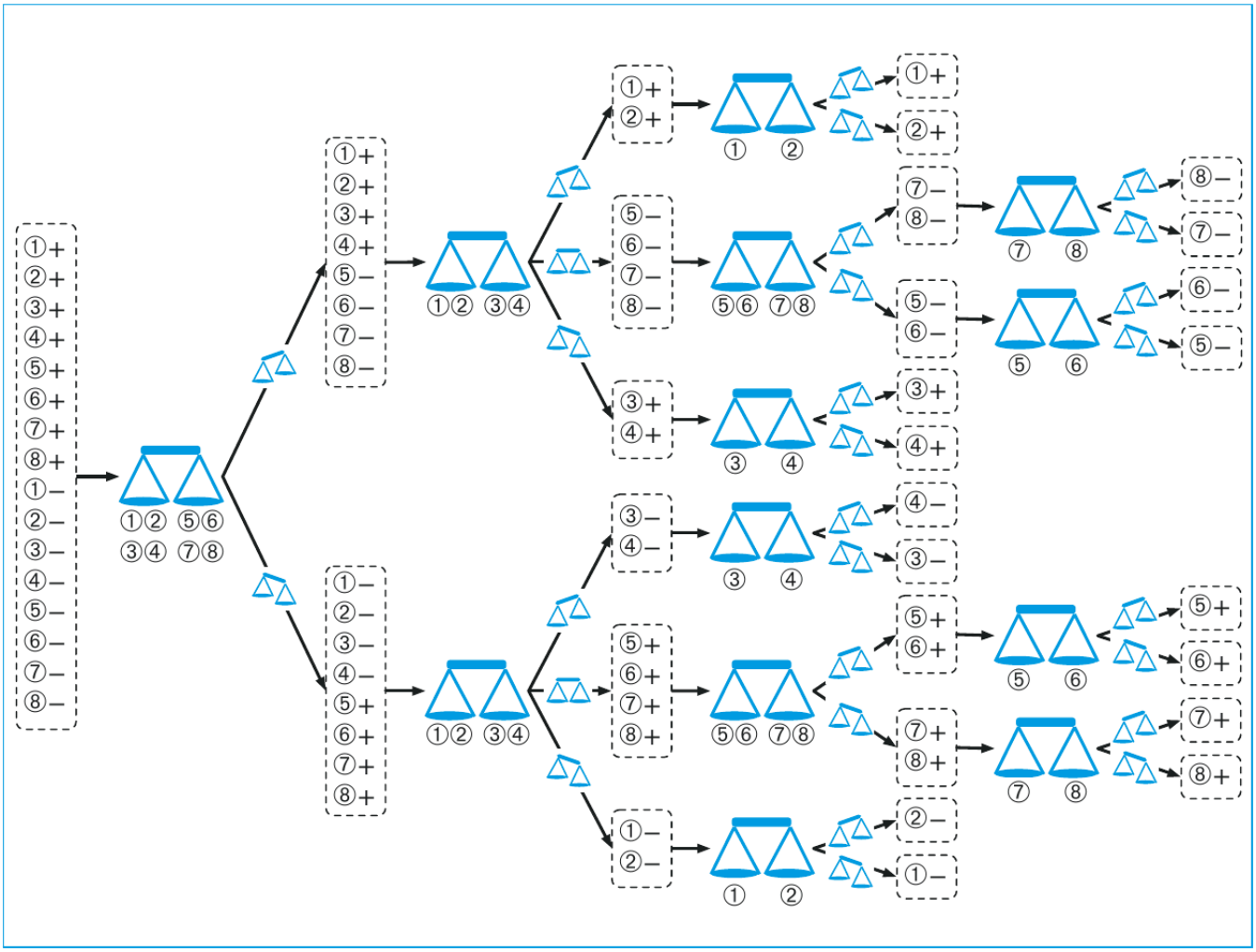


Abbildung 3.42: Mögliche Lösung des Kugelproblems mit 8 Kugeln

- Schlägt die Waage im ersten Schritt nach links aus, enthält die linke Vierergruppe eine schwerere oder die rechte eine leichtere Kugel; nach rechts entsprechend umgekehrt.
- Zu Beginn gibt es 16 Möglichkeiten (①+ … ⑧+ schwerer, ①− … ⑧− leichter). Der erste Wiegeschritt reduziert sie von 16 auf 8: Informationsgewinn 1 Bit.

Verbesserung: Die Waage liefert eine von drei Antworten (links, rechts, Gleichgewicht, Abb. 3.43), nach 3 Wiegeschritten also $3 \cdot 3 \cdot 3 = 27$ unterscheidbare Konstellationen – prinzipiell könnten 3 Schritte reichen.

Informationstheoretisch: Eine Strategie ist optimal, wenn sie in jedem Schritt möglichst viel Information liefert, d. h. die möglichen Ausgänge eines Wiegevorgangs ein möglichst ausgeglichenes Maß an Information liefern – nur so entsteht ein balancierter Baum mit etwa gleich langen Pfaden. Bei der Ad-hoc-Strategie schlägt die Waage im ersten Schritt mit Wahrscheinlichkeit je $\frac{1}{2}$ nach links oder rechts aus, bleibt aber mit Wahrscheinlichkeit 0 in Ruhe; die Information „gleich schwer“ wird verschenkt.

Optimal: $p(\text{links}) = p(\text{rechts}) = p(\text{Gleichgewicht}) = \frac{1}{3}$. Jede Antwort liefert dann $\log_2 3 \approx 1{,}585$ Bit. Eine von 16 Möglichkeiten zu kennen entspricht 4 Bit, also sind mindestens

$$\left\lceil \frac{4}{1{,}585} \right\rceil = 3$$

Wiegeschritte notwendig. Eine nach diesem Grundsatz konstruierte Strategie (Abb. 3.44, erster Schritt ①②③ gegen ④⑤⑥) löst das Problem in 3 Schritten.

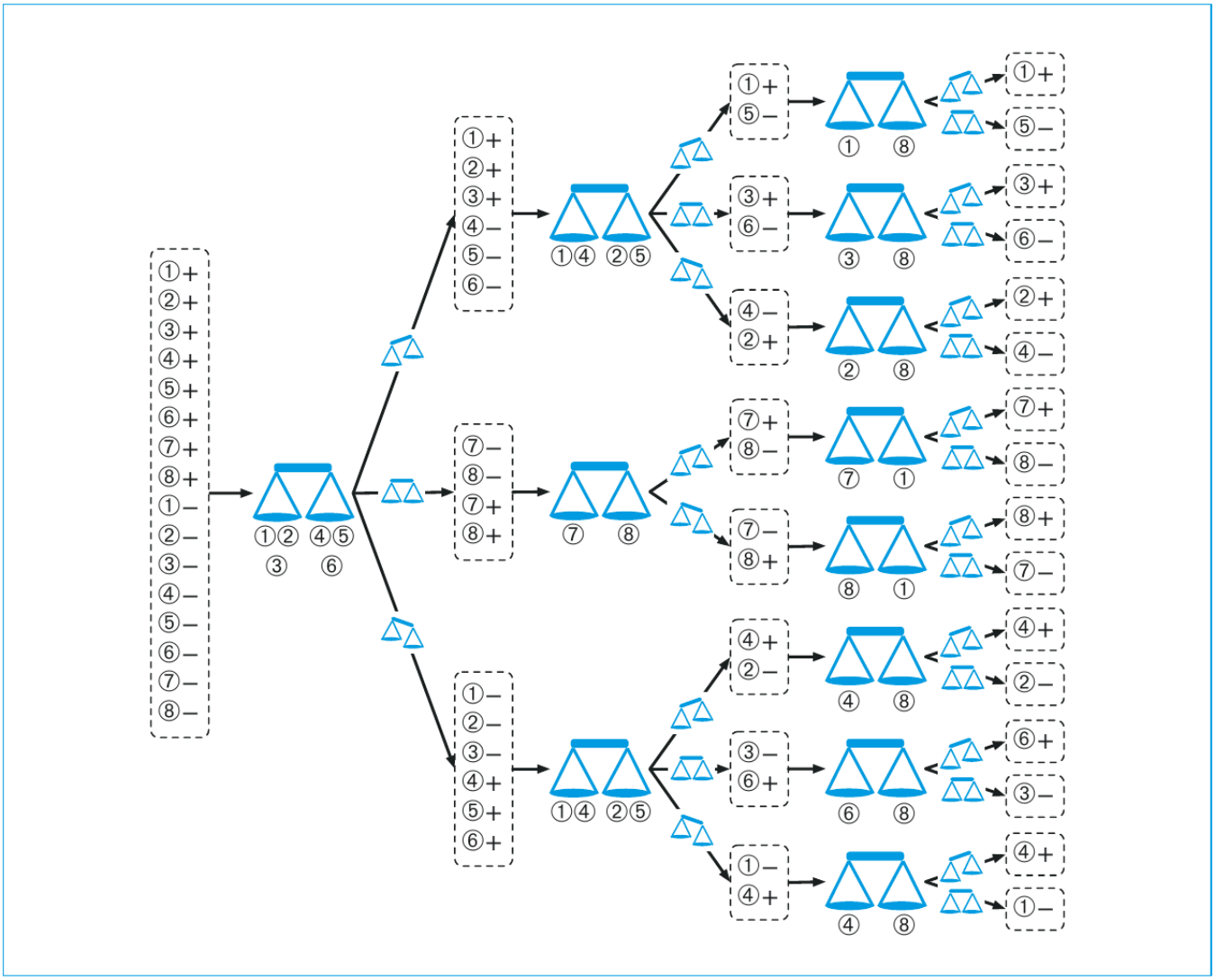


Abbildung 3.44: Das Kugelproblem mit 8 Kugeln lässt sich in 3 Wiegeschritten lösen.